Programming Review

Pointers, OOP, Templates, Exceptions, Recursion

✉

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on April 10, 2026

Outline

1. Why this review
2. Pointers
3. OOP
4. Templates
5. Exceptions
6. Recursion
7. Summary

Why this review

Outline
2 Why this review
Pointers
OOP
Templates
Exceptions
Recursion
Summary

C++ We Need for Data Structures

- **What.** This deck reviews the C++ tools used to *implement* structures.
- **In DSA.** Lists, trees, heaps, and graphs are built from pointers, objects, templates, and recursive definitions.

Tool	Typical use later
pointers	node links, dynamic arrays, aliases
OOP	a structure as a class with a lifetime
templates	`List<T>`, not a new class per type
exceptions	a broken contract (bad index, empty pop)
recursion	walk a list tail or a left/right tree

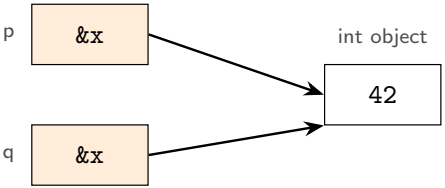
Pointers

Outline
Why this review
4 Pointers
Address vs object
Forms of pointers
Pointer to pointer
Pointers and const
Initialization
Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

What a Pointer Is

- **What.** A pointer is a variable that stores an **address**. The object lives at that address.
- **In DSA.** A list node holds a pointer to the next node; a tree node holds pointers to children; a dynamic array is a pointer to a heap block.
- **Principle.** `p` is the address; `*p` is the object. Assignment `q = p` copies the address, not a new object.

Pointer vs Object — Code



alias: p and q name the same object

```
int x = 42;
int* p = &x;
int* q = p; // alias: *p and *q
    are the same object
*q = 7; // x becomes 7
```

- After `q = p`, `*q = 7` changes `x`.
- **Notes.** Passing a pointer by value copies the address. To change the caller's pointer itself, you need `T**` (or a reference to a pointer).

Programming Review
Outline
Why this review
Pointers
6 Address vs object
Forms of pointers
Pointer to pointer
Pointers and const
Initialization
Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

Forms of Pointer

- **What.** The type of a pointer says *what kind of object* the address refers to, and whether the pointer may be empty or stale.
- **In DSA.** You will read `Node*`, `Node**` (a head you can reseat), and `nullptr` (empty list / missing child).
- **Principle.** Name the form before you dereference it.

Form	Meaning
`T*`	address of a `T`, or of the first array element
`T**`	address of a `T*` — can change that `T*`
`nullptr`	empty on purpose
dangling	the object has already been released

Programming Review
Outline
Why this review
Pointers
Address vs object
7 Forms of pointers
Pointer to pointer
Pointers and const
Initialization
Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

Forms of Pointer — Notes

- An array name decays to a pointer to the first element.
- `nullptr` is a value you chose. Dangling is a value you must not use.
- **Notes.** We do not treat `void*` as the default in this course. Keep types so the compiler can check them.

Outline
Why this review
Pointers
Address vs object
8 Forms of pointers
Pointer to pointer
Pointers and const
Initialization
Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

Pointer to Pointer

- **What.** A `T**` stores the address of a `T*`.
- **In DSA.** Inserting at the front of a list must change the caller's `head`. A function that receives only `Node*` by value cannot do that.
- **Principle.** `*pp` is the inner pointer; `**pp` is the object. Writing `*pp = ...` reseats the inner pointer.

Programming Review

Outline

Why this review

Pointers

Address vs object

Forms of pointers

9 Pointer to pointer

Pointers and const

Initialization

Null and dangling

new and delete

Pointer arithmetic

OOP

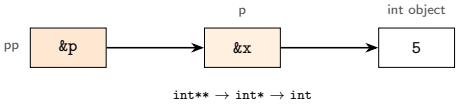
Templates

Exceptions

Recursion

Summary

Pointer to Pointer — Code

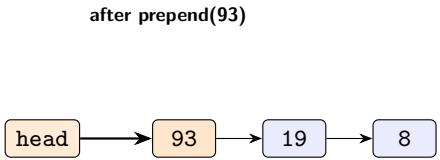



```
int x = 5;
int* p = &x;
int** pp = &p;
**pp = 9; // x becomes 9
```

- *pp is the pointer p; **pp is the int.

Outline
Why this review
Pointers
Address vs object
Forms of pointers
10 Pointer to pointer
Pointers and const
Initialization
Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

Pointer to Pointer — In a List




```
struct Node { int key; Node* next; };

void prepend(Node** head, int key) {
    Node* n = new Node{key, *head};
    *head = n; // changes the caller's
         pointer
}
```

- **Notes.** `prepend` writes through `head`. Forgetting `*` and writing `head = n` would only change the local copy.

Pointers and `const`

- **What.** `const` on a pointer declaration can protect the object, the pointer, or both.
- **In DSA.** A function that only *reads* a structure should take `const T*` (or `const T&`) so it cannot mutate nodes by accident.
- **Principle.** Read from `*`: `const` on the left locks the object (`*p`); `const` on the right locks the pointer (`p`).

Declaration	Reseat `p`?	Write `*p`?
`T* p`	yes	yes
`const T* p`	yes	no
`T* const p`	no	yes
`const T* const p`	no	no

- Quick check: `T const * p` is the same as `const T* p`. Landmark is `*`, not the word `T`.

Programming Review
Outline
Why this review
Pointers
Address vs object
Forms of pointers
Pointer to pointer
12 Pointers and const
Initialization
Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

Pointers and `const` — Code

```
int x = 5;
const int* pc = &x; // const left of *: cannot write *pc
int* const cp = &x; // const right of *: cannot reseat cp
// int const* q = &x; // same as const int*
// *pc = 1; // error
// cp = nullptr; // error
```

- `const int* pc` may point at a different `int` later; it may not assign through `*pc`.
- `int const* pc` is the same declaration (const still left of `*`).
- `int* const cp` always points at the same `int`; it may still write `*cp`.
- **Notes.** `const` on the method (`int size() const`) means this object is not mutated. That is not a const pointer member.

Programming Review

Outline

Why this review

Pointers

Address vs object

Forms of pointers

Pointer to pointer

13 Pointers and const

Initialization

Null and dangling

new and delete

Pointer arithmetic

OOP

Templates

Exceptions

Recursion

Summary

Always Initialise Pointers

- **What.** A pointer holds an address. If you never set it, that address is garbage.
- **In DSA.** An uninitialized `Node* next` looks like a valid link. Walking it is undefined behaviour, not "a random node".
- **Principle.** Every pointer is either `nullptr` or the address of a live object you can name.

Initialization — Code

```
int* p; // uninitialized: do not dereference
int* q = nullptr; // empty on purpose
int* r = new int{3}; // owns a heap object
```

- `p` must not be dereferenced.
- `q` is empty on purpose — test it before use.
- `r` owns a heap object — someone must `delete` it.
- **Notes.** Do not "print `*p` to see what happens". That is UB, not a teaching output.

Programming Review

Outline

Why this review

Pointers

Address vs object

Forms of pointers

Pointer to pointer

Pointers and const

15 Initialization

Null and dangling

new and delete

Pointer arithmetic

OOP

Templates

Exceptions

Recursion

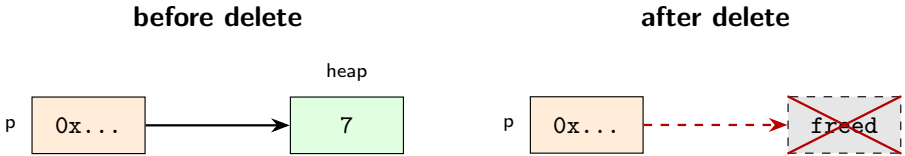
Summary

Null vs Dangling

- **What.** `nullptr` means "no object". Dangling means "there was an object; it is gone".
- **In DSA.** An empty list is `head == nullptr`. After `delete` of a node, any leftover pointer to that node is dangling.
- **Principle.** Never dereference null. Never dereference dangling. After `delete`, reset the pointer if it still lives.

Programming Review
Outline
Why this review
Pointers
Address vs object
Forms of pointers
Pointer to pointer
Pointers and const
Initialization
16 Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

Null vs Dangling — Picture



dangling: do not dereference

Null vs Dangling — Code

```
int* p = new int{7};
delete p;
// p is dangling. Do not read *p (undefined behaviour).
p = nullptr; // safe: mark as empty
```

- `delete` does not change other aliases of the same address.
- **Notes.** We ask "what is wrong / how do we fix it?", not "what number is printed after free?".

Programming Review
Outline
Why this review
Pointers
Address vs object
Forms of pointers
Pointer to pointer
Pointers and const
Initialization
18 Null and dangling
new and delete
Pointer arithmetic
OOP
Templates
Exceptions
Recursion
Summary

new and delete

- **What.** `new` allocates a heap object and returns its address. `delete` releases that object.
- **In DSA.** List/tree nodes and resizable arrays live on the heap so they outlive the function that created them.
- **Principle.** Pair `new` with `delete`, `new[]` with `delete[]`. Exactly one owner releases the block.

Programming Review

Outline

Why this review

Pointers

Address vs object

Forms of pointers

Pointer to pointer

Pointers and const

Initialization

Null and dangling

19 new and delete

Pointer arithmetic

OOP

Templates

Exceptions

Recursion

Summary

new and delete — Code

new T $\leftrightarrow$ delete

new T[] $\leftrightarrow$ delete[]

new T[] $\not\leftrightarrow$ delete

```
int* p = new int{3};
delete p;

int* q = new int[4];
delete[] q; // must match new[]
```

- A borrowed pointer (a visitor, a temporary alias) must not `delete`.
- **Notes.** Leaving a local pointer variable does not free the heap. Only `delete` (or a destructor that calls it) does.

Programming Review

Outline

Why this review

Pointers

Address vs object

Forms of pointers

Pointer to pointer

Pointers and const

Initialization

Null and dangling

20 new and delete

Pointer arithmetic

OOP

Templates

Exceptions

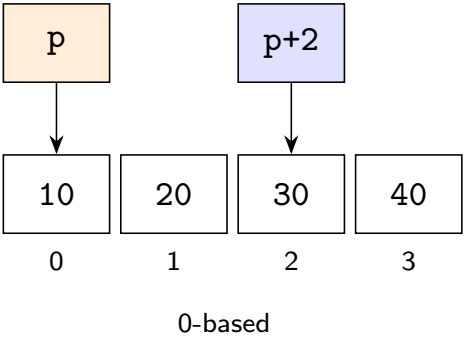
Recursion

Summary

Pointer Arithmetic

- **What.** Adding an integer to a `T*` moves by that many *elements* of type `T`, inside one array.
- **In DSA.** A contiguous buffer (dynamic array, heap) can be scanned this way. A linked list cannot: you follow `next`, you do not add 1 to a node pointer and expect the neighbour.
- **Principle.** Stay inside the same array. `p+1` is not "plus one byte" unless `T` is `char`.

Pointer Arithmetic — Code




```
int a[4] = {10, 20, 30, 40};
int* p = a; // points at a[0]
int* q = p + 2; // points at
    a[2] (same array)
```

- Indexing is 0-based: valid indices are $0, \ldots, n-1$.
- **Notes.** After an array is passed to a function, `sizeof` no longer yields the element count.

OOP

Programming Review
Outline
Why this review
Pointers
23 OOP
Class and object
Constructor and destructor
Copy and assignment
Inheritance
virtual and override
Pure virtual
Templates
Exceptions
Recursion
Summary

Class and Object

- **What.** A class groups data and the operations that preserve its meaning. An object is one instance.
- **In DSA.** A list, stack, or tree is later a class: private representation, public operations (`size`, `push`, ...).
- **Principle.** The client uses the public interface. Fields stay private so invariants can be enforced.

Programming Review

Outline

Why this review

Pointers

OOP

24 Class and object

Constructor and destructor

Copy and assignment

Inheritance

virtual and override

Pure virtual

Templates

Exceptions

Recursion

Summary

Class and Object — Code

```
class IntArray {
private:
    int* data;
    int n;
public:
    int size() const { return n; }
    int get(int i) const { return data[i]; }
};
```

- `data` is a pointer: the array is not "inside" the object bytes the same way `n` is.
- **Notes.** A `struct` may have methods. The issue is not the keyword; it is whether the representation is hidden and whether heap lifetime is handled.

Programming Review
Outline
Why this review
Pointers
OOP
25 Class and object
Constructor and destructor
Copy and assignment
Inheritance
virtual and override
Pure virtual
Templates
Exceptions
Recursion
Summary

Constructor and Destructor

- **What.** The constructor builds a valid object. The destructor undoes acquisitions (heap, files, ...) when the object dies.
- **In DSA.** If a member is a pointer to a node array or a buffer, the constructor `news` it and the destructor `deletes` it.
- **Principle.** Whatever the constructor acquired, the destructor releases. Leaving scope does not free the heap by itself.

Programming Review

Outline

Why this review

Pointers

OOP

Class and object

26 Constructor and destructor

Copy and assignment

Inheritance

virtual and override

Pure virtual

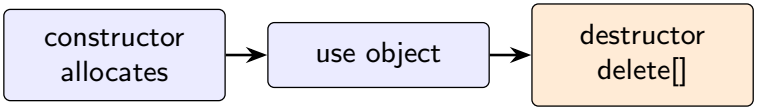
Templates

Exceptions

Recursion

Summary

Constructor and Destructor — Code




```
class IntArray {
    int* data;
    int n;
public:
    IntArray(int n) : data(new int[n]{}), n(n) {}
    ~IntArray() { delete[] data; }
};
```

- **Notes.** If you forget the destructor, the pointer member is destroyed but the array it pointed to leaks.

Programming Review
Outline
Why this review
Pointers
OOP
Class and object
27 Constructor and destructor
Copy and assignment
Inheritance
virtual and override
Pure virtual
Templates
Exceptions
Recursion
Summary

Copy When a Member Is a Pointer

- **What.** The default copy copies each member bitwise. For a pointer, that copies the *address*.
- **In DSA.** Two list objects that share one chain of nodes will both try to `delete` it. That is a double-free.
- **Principle.** If you copy an object that owns a pointer, copy the *data* (deep copy), or forbid copying.

Programming Review

Outline

Why this review

Pointers

OOP

Class and object

Constructor and destructor

28 Copy and assignment

Inheritance

virtual and override

Pure virtual

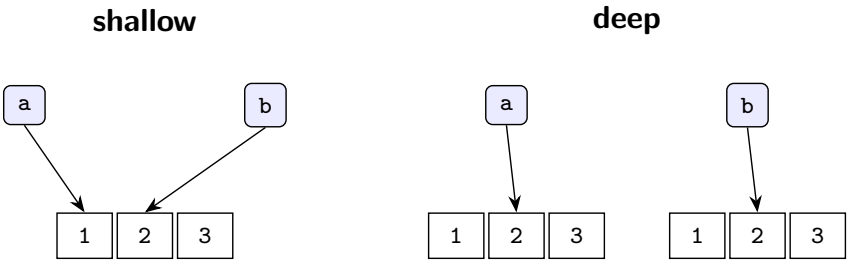
Templates

Exceptions

Recursion

Summary

Default Copy Is Shallow




```
IntArray a(3);
IntArray b = a; // default copy: two objects, one heap buffer
// destroying a then b => double-free (undefined behaviour)
```

- **Notes.** The bug shows up when the second destructor runs, not at the copy line.

Programming Review
Outline
Why this review
Pointers
OOP
Class and object
Constructor and destructor
29 Copy and assignment
Inheritance
virtual and override
Pure virtual
Templates
Exceptions
Recursion
Summary

Copy Constructor and Assignment — Code

```
IntArray(const IntArray& o) // copy constructor
    : data(new int[o.n]), n(o.n) { /* copy elements */ }

IntArray& operator=(const IntArray& o) { // copy assignment
    if (this == &o) return *this;
    delete[] data;
    n = o.n;
    data = new int[n];
    /* copy elements */
    return *this;
}
```

- Copy constructor: the new object does not yet own storage — allocate, then copy elements.
- Assignment: this object already owns storage — release it, then copy. Guard `a = a`.
- **Notes.** We do not teach move / Rule of Five here. If you write one of copy ctor, copy assign, destructor, review the others.

Outline
Why this review
Pointers
OOP
Class and object
Constructor and destructor
30 Copy and assignment
Inheritance
virtual and override
Pure virtual
Templates
Exceptions
Recursion
Summary

Inheritance

- **What.** A derived class *is-a* base class: it reuses the base interface and may add data or operations.
- **In DSA.** Different backings (array vs linked nodes) can share a common sequence interface. We do not implement List operations on this slide.
- **Principle.** Use **public** inheritance for is-a. A derived object may be used where a base is expected.

Programming Review

Outline

Why this review

Pointers

OOP

Class and object

Constructor and destructor

Copy and assignment

31 Inheritance

virtual and override

Pure virtual

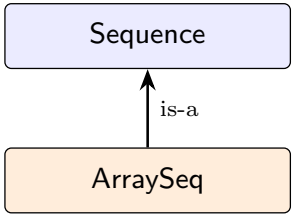
Templates

Exceptions

Recursion

Summary

Inheritance — Code




```
class Sequence { /* shared operations */ };
class ArraySeq : public Sequence { /*
    array backing */ };

ArraySeq a;
Sequence& s = a; // an ArraySeq is a
    Sequence
```

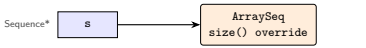
- `Sequence& s = a` does not copy the object; it names the same `ArraySeq` as a `Sequence`.
- **Notes.** Same method name in base and derived is *not* automatically polymorphic. That needs `virtual`.

Programming Review
Outline
Why this review
Pointers
OOP
Class and object
Constructor and destructor
Copy and assignment
32 Inheritance
virtual and override
Pure virtual
Templates
Exceptions
Recursion
Summary

Polymorphism

- **What.** Polymorphism: the call uses the *object's* type, not the type of the pointer/reference.
- **In DSA.** `Sequence* s` may point at an array-backed or node-backed object. `s->size()` should run the version that matches the object.
- **Principle.** Write `virtual` on the base method. Write `override` on the derived method. If you `delete` through a base pointer, the destructor must be virtual.

virtual and override — Code



`s->size()` uses the object, not the pointer type

```
class Sequence {
public:
    virtual int size() const { return 0; }
    virtual ~Sequence() {}
};
class ArraySeq : public Sequence {
    int n;
public:
    int size() const override { return n; }
};
Sequence* s = new ArraySeq;
s->size(); // ArraySeq::size
delete s; // virtual destructor
```

- Without `virtual`, `s->size()` would call `Sequence::size`.
- **Notes.** We do not draw compiler internals. Remember the rule: virtual dispatch follows the object.

Programming Review
Outline
Why this review
Pointers
OOP
Class and object
Constructor and destructor
Copy and assignment
Inheritance
34 virtual and override
Pure virtual
Templates
Exceptions
Recursion
Summary

Pure Virtual Methods

- **What.** = 0 marks a method that has no body in this class. Derived classes must override it.
- **In DSA.** The interface "a sequence has a size" can be stated without choosing an array or a chain of nodes.
- **Principle.** A class with a pure virtual method is **abstract**: you cannot create an object of that type.

Outline
Why this review
Pointers
OOP
Class and object
Constructor and destructor
Copy and assignment
Inheritance
virtual and override
35 Pure virtual
Templates
Exceptions
Recursion
Summary

Pure Virtual — Code

```
class Sequence {
public:
    virtual int size() const = 0; // pure virtual
    virtual ~Sequence() {}
};
// Sequence s; // error: abstract
// a derived class must override size()
```

- `Sequence s;` does not compile.
- A concrete derived class must override every pure method.
- **Notes.** Keep a virtual destructor on the abstract base so `delete` through `Sequence*` is defined.

Programming Review
Outline
Why this review
Pointers
OOP
Class and object
Constructor and destructor
Copy and assignment
Inheritance
virtual and override
36 Pure virtual
Templates
Exceptions
Recursion
Summary

Templates

Outline
Why this review
Pointers
OOP
37 Templates
Exceptions
Recursion
Summary

Templates

- **What.** A template is a blueprint. The compiler generates a class (or function) for each element type you use.
- **In DSA.** We want `List<int>` and `List<double>`, not `IntList` plus `DoubleList` with copied bugs.
- **Principle.** Parameterise the element type with `template<typename T>`. Do not hardcode `int` in the structure.

Templates — Code

```
template<typename T>
class Array {
    T* data;
    int n;
public:
    Array(int n) : data(new T[n]{}), n(n) {}
    ~Array() { delete[] data; }
};
Array<int> a(8);
Array<double> b(4);
```

- `Array<int>` and `Array<double>` are different types, from one piece of source.
- **Notes.** `T` does not remove ownership. If a member is `T*`, you still write destructor, copy, and assignment.

Exceptions

Outline
Why this review
Pointers
OOP
Templates
40 Exceptions
Recursion
Summary

try / catch

- **What.** An exception is a way to abandon the current work and report a failure that the caller must handle.
- **In DSA.** A bad index, pop on an empty stack, or a broken precondition can `throw` instead of returning a dummy value.
- **Principle.** `throw` starts unwinding. `catch` handles a type. Catch by `const` reference.

try / catch — Code

```
int at(const int a[], int n, int i) {
    if (i < 0 || i >= n)
        throw std::out_of_range("index");
    return a[i];
}
try {
    int x = at(a, n, i);
} catch (const std::out_of_range& e) {
    // handle, then continue or rethrow
}
```

- `throw` is not a second normal return path for "not found" on every search.
- While unwinding, destructors of local objects still run (the pointer-member destructor can still `delete`).
- **Notes.** If nothing catches, the program terminates. Do not catch only to ignore and continue blindly.

Programming Review

Outline

Why this review

Pointers

OOP

Templates

42 Exceptions

Recursion

Summary

Recursion

Outline
Why this review
Pointers
OOP
Templates
Exceptions
43 Recursion
Recursive structures
Base case and recursive call
The call stack
Hanoi, same pattern
Summary

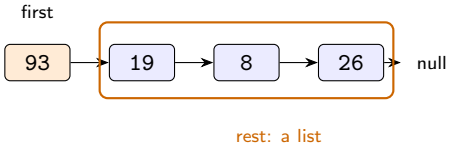
Why Recursion Fits Data Structures

- **What.** A definition is recursive when the object is described using a smaller object of the same kind.
- **In DSA.** A list is a first element plus the rest of the list (or empty). A binary tree is a root plus a left tree and a right tree (or empty).
- **Principle.** Match the code to the definition: base case for empty, recursive call on the rest / the children.

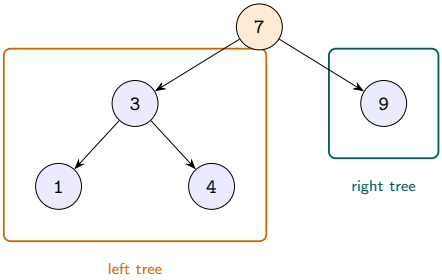
Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
44 Recursive structures
Base case and recursive call
The call stack
Hanoi, same pattern
Summary

Recursive List and Tree

list = first element + rest of the list



tree = root + left tree + right tree



Recursive Structures — Code

```
struct Node { int key; Node* next; };

int length(Node* p) {
    if (p == nullptr) return 0; //
        base: empty list
    return 1 + length(p->next); //
        first + rest
}
```

```
struct TNode { int key; TNode* left;
    TNode* right; };

int size(TNode* t) {
    if (t == nullptr) return 0; //
        base: empty tree
    return 1 + size(t->left) +
        size(t->right);
}
```

- Empty list / empty tree is `nullptr`: that is the base case.
- **Notes.** This is not a List-ADT exam. It shows why the later chapters recurse on `next`, `left`, and `right`.

Programming Review
Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
46 Recursive structures
Base case and recursive call
The call stack
Hanoi, same pattern
Summary

Base Case and Recursive Call

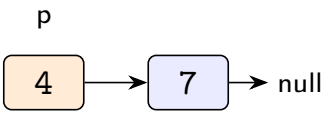
- **What.** A recursive algorithm stops on a **base** instance and calls itself on a **smaller** instance.
- **In DSA.** Sum the keys of a singly linked list of `int`: empty list contributes 0; otherwise first key plus the sum of the rest.
- **Principle.** (1) stopping case, (2) how the instance shrinks, (3) add *after* the inner call returns.

$$\mathrm{sum}(p) = \begin{cases} 0 & p = \text{null} \quad \text{(base)} \\ p\texttt{->key} + \mathrm{sum}(p\texttt{->next}) & \text{otherwise (recursive call)} \end{cases}$$

Programming Review
Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
Recursive structures
47 Base case and recursive call
The call stack
Hanoi, same pattern
Summary

Sum a List — Code

sum: $4 + 7 + 0 = 11$




```
struct Node { int key; Node* next; };

int sum(Node* p) {
    if (p == nullptr) return 0; // base
        case
    return p->key + sum(p->next); //
        recursive call
}
```

- Base: `p == nullptr` ⇒ return `0`. No further call.
- Recursive call: `sum(p->next)` — a shorter list.
- `p->key + ...` is not added until that call has returned.

Programming Review
Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
Recursive structures
48 Base case and recursive call
The call stack
Hanoi, same pattern
Summary

An Activation Record

one activation record

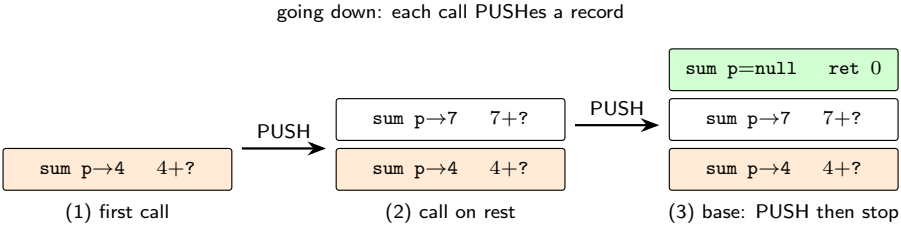
```
sum
parameter p → node 4
waiting:  4 + ?
resume after the inner call
```

PUSH this record when `sum` is called.
The `?` is filled when the inner call returns, then POP.

- **What.** Each call gets its own record: the pointer p, and a hole ? for the inner result.
- **Principle.** The runtime keeps records on a **stack**: PUSH on call, POP on return.
- The live depth is how many records sit on the stack at once.

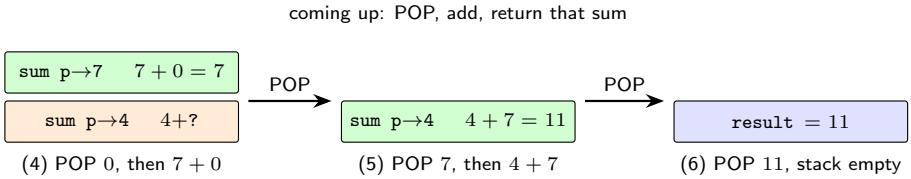
Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
Recursive structures
Base case and recursive call
49 The call stack
Hanoi, same pattern
Summary

Going Down — PUSH



- Each recursive call PUSHes a new record on top. The caller waits.
- Step (3) is the base case: p is null, so the record returns 0 immediately.

Coming Up — POP



- POP the top record and send its return value into the ? of the caller.
- Then compute `p->key + ?` and return that sum — which is the next POP.
- **Notes.** If the base case is missing, PUSH never stops (stack overflow).

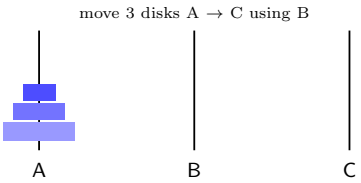
Programming Review
Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
Recursive structures
Base case and recursive call
51 The call stack
Hanoi, same pattern
Summary

Towers of Hanoi — Same Pattern

- **What.** Move n disks: move $n-1$, move 1, move $n-1$.
- **In DSA.** Same recipe as "solve a smaller instance, do one step, solve a smaller instance". Enough to recognise it; we do not drill every disk move.
- **Principle.** Base: one disk. General: two recursive calls plus one move.

Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
Recursive structures
Base case and recursive call
The call stack
52 Hanoi, same pattern
Summary

Hanoi — Code




```
void move(int n, char src, char dst, char
     aux) {
    if (n == 1) { printMove(src, dst);
         return; }
    move(n - 1, src, aux, dst);
    move(1, src, dst, aux);
    move(n - 1, aux, dst, src);
}
```

- **Notes.** How many calls, and $T(n) = 2T(n-1) + 1$, belong to the complexity deck.

When Recursion Helps

- Prefer it when the data (or the problem) is self-similar: list tail, left/right tree, Hanoi.
- The machine pays with activation records. Deep trees need deep stacks.
- Recursion is not automatically faster or shorter in running time.
- **Notes.** Naive overlapping calls (Fibonacci) are analysed next; we do not rewrite them as $2T(n-1)$ here.

Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
Recursive structures
Base case and recursive call
The call stack
54 Hanoi, same pattern
Summary

Summary

Outline
Why this review
Pointers
OOP
Templates
Exceptions
Recursion
55 Summary

Summary

- Pointers: address vs object; initialise; null vs dangling; `T**` to reseat a head.
- OOP: if a member is a pointer, write ctor, dtor, and deep copy / assignment; `virtual`/`override`/`= 0` for a shared interface.
- Templates replace a hardcoded element type; they do not replace ownership.
- `throw`/`catch` report a broken contract; destructors still run while unwinding.
- Recursion: match the structure; base + smaller instance; the runtime uses a stack of frames.
- Next: count the cost of loops and of these recursive calls.